

Result Tips

David Wickes

I was trying to explain the `Result` type in our Kotlin code, as people had been asking me to do a lightning talk on `Result`.

Can You Explain

`Result` In Just

Five Minutes?

Which sounds hard - is it even possible?

No

You cannot

No, it is not possible and so
I'd like to advertise my forthcoming talk

Forthcoming Attraction:

Explain `Result` in

20 Minutes

Where I'll explain `Result` in around twenty minutes. I hope. But anyway

Anyway

~~Explain `Result` in Five Minutes~~

Four? Result Tips

So instead I've gone with some quick tips to help navigate using `Result`. Sure, you may not understand it afterwards, but you might just be able to get away with using it.

tip one

`Result` is never an argument

Now, to be fair, I've never seen this in the SNAPP code, so well done everyone. But I *have* seen it done. And it goes something like this:

Looks something like this

```
val result: Result<Int, Error> =  
    parse(request)  
  
val thingIWant = bizz(result)  
  
// etc..
```

You want to save yourself some time - if the form parsing function returns a `Result`, why not just make the function that needs the content of the form take that same `Result`? Nice and simple.

The problem is that it messes up the code on the inside of the function:

Yeuch

```
fun bizz(r: Result<Int, Error> {  
    if (r is Failure) {  
        return r  
    }  
    val actual = r.value  
  
    // do BUSINESS  
}
```

Now all your business handling logic is getting super paranoid about handling errors in its arguments... as well as handling any other errors that are happening inside. Chaos, I tell you!

Prefer

```
parse(request)  
    .flatMap { n -> bizz(n) }  
  
fun parse(r: Req): Result<Int, Err> //...  
  
fun bizz(n: Int): Result<More, Err> //...
```

`Result` (mostly) lets us handle our errors on the outside, after we've done all the work we need to do, so that our functions *don't* have to worry about whether the last operation went wrong. Chain then together, not by passing the `Result` around, but by using `flatMap` on the `Result`.

Oh yeah, and talking about `flatMap`

tip two

use `map` and `flatMap` *right*

You need to know the difference between `map` and `flatMap`.

They both

Pass the `Success` value through a function

```
(Value) -> ???
```

So they're both the same in one way - they are methods on a `Result` that pass the `Success` value through a function - and change the `Result` in some way.

It's a way of "doing a thing" to a `Result` without having to know what's inside it.

The difference is...

`flatMap` is for

functions that *do* return a `Result`

```
(Value) -> Result<AnotherValue, Err>
```

`flatMap` wants to work with a function that turns your value into another `Result` - like this:

```
fun parse(r: Request): Result<Int, Err>

fun bizz(n: Int): Result<MoreStuff, Err>

val x: Result<MoreStuff, Err> =
    parse(request)
        .flatMap { n -> bizz(n) }
```

So you see that both of our functions here return a `Result`, so when we want to chain them together, we use `flatMap`.

The value at the end of it all is actually the last `Result` returned from the last function you `flatMap`ed through - so in this case it's the `Result` from `bizz` - the `Result<MoreStuff, Err>`.

Buuuut `map`...

`map` is for

functions that *don't* return a `Result`

```
(Value) -> AnotherValue
```

...`map` is for functions that don't return a `Result`.

```
fun parse(r: Request): Result<Int, Err>

fun toString(n: Int): String

val x: Result<String, Err> =
    parse(request)
        .map { n -> toString(n) }
```

So you can see that when we want to chain `toString` onto the end of our `parse`, because `toString` doesn't return a result we use `map`.

We still end up with a `Result`, but it's now a result of a `String` - the `map` changed the type of `Result<Int, Err>` to a `Result<String, Err>`. Fun right.

On the subject of what a function returns...

tip three

Only Operations That Can Fail Should Return A `Result`

Which sounds kinda obvious when I say it out loud, but, believe me, this happens.

It's usually a consequence of `flatMap` addiction

`flatMap` addiction

```
parse(request)
    .flatMap { doAThing(it) }
    .flatMap { doAnotherThing(it) }
    .flatMap { moreStuffs(it) }
    .flatMap { oooohMapMe(it) }
    .flatMap { addOne(it) }
    .flatMap { flatMapThirst(it) }
    .flatMap(::lookMaReference)
    //.. etc
```

Which is all very clean and nice, and consistent - `flatMap`, `flatMap`, `flatMap`... *but!*
But, look at that middle function, `addOne`. It probably looks like this when you look inside:

bad `addOne`

```
fun addOne(n: Int): Result<Int, Err> =  
    Success(n + 1)
```

We're wrapping up the result of `addOne` in... well, in a `Result`. But `addOne` can't fail - it'll always be successful. And if it's always successful _then why the hell is it a `Result`?

"But what's the harm?"

meaning is important

Sure, you may say "what's the harm, really, it still works right?" - but remember you're not just writing code to make the computer work. Your code reveals intention and meaning to the both you and the next people to work on it. If I see a function that returns a `Result`, then I expect it to be able to fail. And I will probably try to handle that failure, or think about why it fails, or wonder if I should be using it if can fail... even though it really does no such thing.

Happily we can easily rewrite `addOne`:

good `addOne`

Which is pretty easy to read, right?

```
fun addOne(n: Int): Int = n + 1
```

and in use:

```
parse(request)  
    .flatMap { doAThing(it) }  
    .flatMap { doAnotherThing(it) }  
    .map { addOne(it) }  
    .flatMap { flatMapThirst(it) }  
    //.. etc
```

you just use `map` instead of `flatMap`.

So, really, what you need to know is

`map` is for when things can't go wrong, and `flatMap` is for when things *can* go wrong.

`map` when things can't go wrong

`flatMap` when things *can* go wrong

tip four

Pragmatism

The chain of `flatMap`s we've just seen is pretty ideal when you're dealing with the transformation of a single value *though* a series of fallible operations. It's lovely. It

Things get... well, they get pretty awful when you need to start collecting lots of different values from `Results` to use together.

From [here](#)

One single expression

```
submissionRepository
    .getSubmission(submissionId)
    .flatMap { sub ->
        jRepo.getJournal(sub.journalId)
            .map { journal ->
                sub to journal
            }
    }
```

Here we have a pretty simple idea - get a submission from a `submissionId`, get the journal using the `journalId` on the submission, and then return them both as a pair.

That *sounds* simple - so why does it look so god-awful?

Well, other, more functional programming languages like Scala and Haskell (and, um, C#) have a special notation for collecting up a series of values - like submission and journal above - without showing you all the nested lambdas that are *really* going on under the hood.

Well, Kotlin doesn't. So you've got limited options.

One - live with this above. This is important if you can't survive without making every function you write into a single expression. Otherwise, you're just being wilfully painful for no reason.

A better solution would be to assign variables and take advantage of early returns:

Better (there is no Best)

```
val sub = submissionRepository
```

```

        .getSubmission(submissionId)
        .onFailure { return it }

    val journal = jRepo
        .getJournal(sub.jFlowId)
        .onFailure { return it }

    return Success(sub to journal)

```

This looks a bit awkward on a slide, but it's not as bad as all that when you see it in the code: `onFailure` 'unwraps' the success - it becomes a `T` not a `Success<T>` - if it's a success, and gives you a lambda to handle the `Failure` and here we handle the failure just by returning it early.

We do the same thing for the journal after the submission, and then, when we've got hold of our "real" journal and submission, we can pair them up - and "re-wrap" them in a `Success`.

Or Find Your Own Way

Like I said, there are other ways of handling this - mainly to do with collecting up the values you want into some structure and using them in the end - take a look at the Anura code and I'm sure you'll see some examples.

I recommend using the `onFailure` - `return` pattern though.

Just don't do this

```

submissionRepository
    .getSubmission(submissionId)
    .flatMap { sub ->
        jRepo.getJournal(sub.journalId)
            .map { journal ->
                sub to journal
            }
    }

```

Summary

1. Don't use `Result` as an argument
2. `map` things that can't fail
3. `flatMap` things that can fail
4. Don't return `Result` if you can't fail
5. early `return` your `Failures` when collecting `Results`

Questions